

UNIwersytet Rzeszowski
Wydział Nauk Ścisłych i Technicznych
Instytut Informatyki



Oleksii Nawrocki, Tomasz Nowak
131400, 131478

Informatyka

System paczkomatów i logistyki - FasterPost

Praca projektowa

Praca wykonana pod kierunkiem
mgr inż. Marcin Mrukowicz

Rzeszów 2026

Spis treści

1. Wstęp i analiza biznesowa	8
1.1. Opis świata rzeczywistego i cel systemu	8
1.2. Analiza rozwiązań istniejących (SOTA)	8
1.2.1. InPost	8
1.2.2. DPD Pickup	8
1.2.3. Orlen Paczka	8
1.3. Wymagania funkcjonalne	9
1.4. Wymagania нефункционалне	10
2. Analiza systemowa i specyfikacja wymagań	11
2.1. Charakterystyka aktorów systemu	11
2.2. Model przypadków użycia	11
2.3. Szczegółowy opis scenariuszy biznesowych	11
2.3.1. Proces nadania przesyłki	13
2.3.2. Obsługa logistyczna i kurierska	13
2.3.3. Zarządzanie i administracja	13
3. Architektura i projektowanie techniczne	14
3.1. Architektura systemu	14
3.1.1. Diagram klas	14
3.2. Dobór technologii (Technology Stack)	15
3.2.1. Warstwa serwerowa (Backend)	15
3.2.2. Warstwa prezentacji (Frontend)	15
3.2.3. Konteneryzacja i środowisko (Docker)	15
3.2.4. Integracja płatności (Stripe)	16
3.3. Projekt warstwy danych (ORM)	16
3.3.1. Moduł Użytkowników i Autoryzacji (Accounts)	16
3.3.2. Moduł Paczek (Packages)	16
3.3.3. Moduł Logistyki (Logistics)	17
3.3.4. Moduł Infrastruktury (Postmats)	17
3.3.5. Diagram ERD	17
3.4. Struktura aplikacji klienta (Frontend)	19
4. Realizacja logiki logistycznej i algorytmów	20
4.1. Logistyka krajowa: Implementacja problemu CVRP	20
4.1.1. Heurystyka Najbliższego Sąsiada (Nearest Neighbor)	20
4.2. Logistyka lokalna: Ostatnia mila i TSP	20
4.2.1. Klasteryzacja i podział na strefy	21
4.2.2. Teoretyczne ujęcie problemu komiwojażera (TSP)	21

4.2.3. Implementacja algorytmu TSP i formuła Haversine'a	21
4.3. Zarządzanie skrytkami i weryfikacja dostępności.....	22
4.3.1. Algorytm alokacji (Allocation Phase)	22
5. Planowanie, testowanie i wdrożenie	23
5.1. Harmonogram realizacji projektu.....	23
5.2. Zapewnienie jakości i testy.....	23
5.2.1. Macierz Śledzenia Wymagań (RTM)	24
5.2.2. Analiza wyników testów	24
5.3. Instrukcja wdrożenia i uruchomienia	25
5.3.1. Wymagania wstępne	25
5.3.2. Konfiguracja zmiennych środowiskowych	25
5.3.3. Uruchomienie kontenerów.....	25
5.3.4. Inicjalizacja danych (Seeding)	26
5.4. Podsumowanie wdrożenia	26
6. Podsumowanie i wnioski.....	27
6.1. Wnioski końcowe i ocena realizacji celów.....	27
6.2. Kierunki dalszego rozwoju (Future Work)	27
6.2.1. Komunikacja w czasie rzeczywistym (WebSockets)	27
6.2.2. Zastosowanie Sztucznej Inteligencji (AI)	28
6.2.3. Migracja do architektury mikroserwisowej	28
7. Projektowanie interfejsu użytkownika dla systemu FastPost Express	29
7.1. Założenia projektowe i strategia UI/UX.....	29
7.2. System autentykacji i onboarding	29
7.3. Proces autentykacji i rejestracji.....	29
7.4. Panele użytkownika i funkcjonalności	30
7.4.1. Pulpit i obsługa przesyłek - użytkownik indywidualny	30
7.4.2. Panel biznesowy	31
7.5. Zarządzanie systemem i logistyka	32
7.6. Elementy pomocnicze	33
7.7. Podsumowanie projektowe	34
8. Rejestr wykorzystania narzędzi AI.....	35
8.1. Rejestr wykorzystania narzędzi AI - Gemini.....	35
8.2. Implementacja heurystyki Nearest Neighbor dla problemu CVRP.....	37
8.3. Projekt bazy danych systemu kurierskiego w Django ORM.....	38
8.4. Konfiguracja Docker Compose dla stosu Django, Next.js, PostgreSQL i Redis.....	40
8.5. Grupowanie punktów GPS za pomocą algorytmu K-Means.....	41
8.6. Bezpieczne uwierzytelnianie w DRF z użyciem ciasteczek HttpOnly	42
Bibliografia	45
Spis rysunków	46
Spis tabel.....	47

Spis listingów	48
Streszczenie.....	49
Oświadczenie studenta o samodzielności pracy.....	50

1. Wstęp i analiza biznesowa

1.1. Opis świata rzeczywistego i cel systemu

System **FasterPost** stanowi zintegrowaną platformę logistyczną, której głównym celem jest automatyzacja procesów nadawania, transportu i odbioru przesyłek w sieci skrytek paczkomatowych. Interakcja z systemem rozpoczyna się w momencie, gdy użytkownik – niezależnie czy jest to klient indywidualny, czy biznesowy – korzysta z aplikacji webowej w celu wypełnienia formularza nadawczego i realizacji płatności online.

Po pomyślnej transakcji system natychmiastowo rezerwuje odpowiednią skrytkę, umożliwiając nadawcy jej otwarcie za pomocą aplikacji, a następnie umieszczenie paczki wewnątrz maszyny. Zmiana statusu przesyłki następuje automatycznie, co uruchamia proces śledzenia w czasie rzeczywistym.

Logistyczna obsługa przesyłki w **FasterPost** opiera się na skoordynowanej pracy kurierów i systemów magazynowych. Kurier lokalny, wyposażony w dedykowany terminal mobilny, odbiera zgromadzone w paczkomatach przesyłki i transportuje je do magazynu lokalnego. Stamtąd trafiają one do sieci transportu międzymagazynowego (Line Haul). Dzięki zaimplementowanym algorytmom optymalizacji tras, paczki sprawnie przemieszczają się między regionami, by finalnie trafić ponownie w ręce lokalnego kuriera, który umieszcza je w docelowym paczkomacie.

Cała infrastruktura zarządzana jest poprzez system dedykowanych paneli, które dostosowują dostępne funkcjonalności do roli zalogowanego użytkownika. Klienci indywidualni skupiają się na pojedynczych operacjach, natomiast użytkownicy biznesowi mają dostęp do narzędzi masowego generowania etykiet. Nad stabilnością czuwa administrator, monitorujący raporty błędów i konfigurację sieci paczkomatów.

1.2. Analiza rozwiązań istniejących (SOTA)

Analiza rynku usług kurierskich (stan na grudzień 2025) pozwala wskazać kluczowe rozwiązania stanowiące punkt odniesienia dla systemu **FasterPost**:

1.2.1. InPost

Lider rynku oferujący sieć ponad 27 000 paczkomatów dostępnych w trybie 24/7. Nadanie i odbiór odbywa się poprzez aplikację mobilną (kod QR) lub kod PIN. Płatności realizowane są w pełni online. System charakteryzuje się zaawansowanym śledzeniem przesyłek oraz wdrażaniem autonomicznych maszyn.

1.2.2. DPD Pickup

Rozwiązanie hybrydowe, łączące automaty paczkowe z siecią punktów partnerskich (sklepy, stacje paliw). Wiele urządzeń to nowoczesne, bezprzewodowe automaty typu SwipBox Infinity. Aplikacja mobilna DPD Mobile umożliwia precyzyjną lokalizację punktów na mapie.

1.2.3. Orlen Paczka

Sieć obejmująca własne automaty oraz punkty partnerskie. Obsługa procesów nadawczo-odbiorczych realizowana jest przez aplikację Orlen Vitay, a standardowy czas na odbiór wynosi 3 dni z możliwością przedłużenia.

1.3. Wymagania funkcjonalne

Na podstawie analizy potrzeb użytkowników zdefiniowano następujące wymagania funkcjonalne systemu, które przedstawiono w Tabeli 1.1.

YY

Y

Tabela 1.1. Zestawienie wymagań funkcjonalnych

Moduł	Wymagania
Obsługa użytkowników	• Rejestracja i logowanie użytkowników (klientów, kurierów, administratorów) poprzez zunifikowany panel. • Edycja danych profilowych (dane kontaktowe, ustawienia bezpieczeństwa). • Weryfikacja tożsamości poprzez wiadomość e-mail.
Obsługa paczek	• Tworzenie przesyłki poprzez interaktywny formularz. • Generowanie etykiety nadawczej z unikalnym numerem śledzenia (Tracking ID). • Wybór paczkomatu nadawczego i docelowego z mapy. • Śledzenie statusu przesyłki w czasie rzeczywistym. • Obsługa płatności online oraz kalkulator kosztów w zależności od gabarytu (S, M, L).
System kuriera	• Automatyczne planowanie optymalnej trasy na podstawie lokalizacji paczkomatów. • Podgląd listy przesyłek przypisanych do bieżącego zlecenia (manifest). • Mobilna obsługa zmiany statusów (np. „Odebrano z magazynu”, „Umieszczono w skrytce”).
Panel administracyjny	• Zarządzanie bazą użytkowników, paczek i kurierów (operacje CRUD). • Podgląd raportów błędów, statystyk dostaw i obciążenia poszczególnych maszyn. • Konfiguracja parametrów globalnych systemu.

Moduł	Wymagania
System paczkomatowy	• Dynamiczna rezerwacja skrytki w momencie opłacenia przesyłki. • Zdalne otwieranie skrytek z poziomu aplikacji. • System powiadomień o gotowości paczki do odbioru.

1.4. Wymagania niefunkcjonalne

Aby zapewnić wysoką jakość usług, system **FasterPost** musi spełniać szereg wymagań jakościowych:

1. **Wydajność i skalowalność:** System powinien być przygotowany na obsługę dużej liczby równoległych zapytań, szczególnie w okresach wzmożonego ruchu (np. święta). Czas odpowiedzi API dla kluczowych operacji nie powinien przekraczać 1 sekundy.
2. **Bezpieczeństwo:**
 - Hasła użytkowników muszą być przechowywane w formie zaszyfrowanej (algorytm bcrypt).
 - Wymagana jest implementacja mechanizmów autoryzacji opartych na rolach.
 - Wszystkie działania administracyjne muszą być logowane.
3. **Użyteczność:** Interfejs użytkownika (Web i Mobile) musi być intuicyjny i responsywny (RWD), dostosowując się do urządzeń mobilnych.
4. **Dokumentacja:** System musi posiadać pełną dokumentację techniczną API oraz instrukcję wdrożenia.

2. Analiza systemowa i specyfikacja wymagań

W niniejszym rozdziale przeprowadzono szczegółową analizę funkcjonalną systemu FasterPost, identyfikując kluczowych aktorów oraz definiując scenariusze ich interakcji z aplikacją. Celem tej analizy było stworzenie precyzyjnego modelu wymagań, który posłużył jako fundament dla późniejszego procesu projektowania architektury oraz implementacji poszczególnych modułów oprogramowania.

2.1. Charakterystyka aktorów systemu

Na podstawie analizy dziedziny problemu oraz wymagań biznesowych wyodrębniono trzy główne grupy użytkowników (aktorów), którzy wchodzą w bezpośrednią interakcję z systemem. Każda z ról posiada odrębny zakres odpowiedzialności oraz dedykowany interfejs użytkownika.

Pierwszym i najważniejszym aktorem jest **Klient**. Jest to użytkownik końcowy, będący inicjatorem procesu logistycznego. W ramach systemu FasterPost rola ta została podzielona na użytkowników niezarejestrowanych, którzy posiadają dostęp jedynie do podstawowych informacji ofertowych oraz śledzenia przesyłek, oraz użytkowników zarejestrowanych. Klient zalogowany dysponuje pełnym wachlarzem funkcjonalności, obejmującym nadawanie przesyłek, zarządzanie książką adresową, dokonywanie płatności online oraz przegląd historii zleceń. Interakcja Klienta z systemem odbywa się głównie poprzez responsywną aplikację internetową.

Drugą kluczową rolę pełni **Kurier**. Jest to pracownik terenowy, odpowiedzialny za fizyczną realizację transportu przesyłek w ramach tzw. "Ostatniej Mili" (Last Mile) oraz transportu międzyhubowego. Aktor ten korzysta ze specjalistycznego modułu aplikacji, dostosowanego do urządzeń mobilnych. Jego głównym zadaniem jest odbiór paczek z maszyn nadawczych, transport do magazynu oraz dystrybucja do paczkomatów docelowych. System wspiera Kuriera poprzez automatyczne wyznaczanie optymalnych tras przejazdu oraz weryfikację dostępności skrzynek w czasie rzeczywistym.

Trzecim aktorem jest **Administrator**. Rola ta posiada najwyższy poziom uprawnień i odpowiada za utrzymanie ciągłości działania systemu. Administrator nie bierze bezpośredniego udziału w procesie przemieszczania paczek, lecz sprawuje nadzór techniczny i operacyjny. Do jego obowiązków należy zarządzanie kontami użytkowników, monitorowanie stanu technicznego infrastruktury (paczkomatów) oraz interwencja w przypadku wystąpienia błędów krytycznych lub reklamacji.

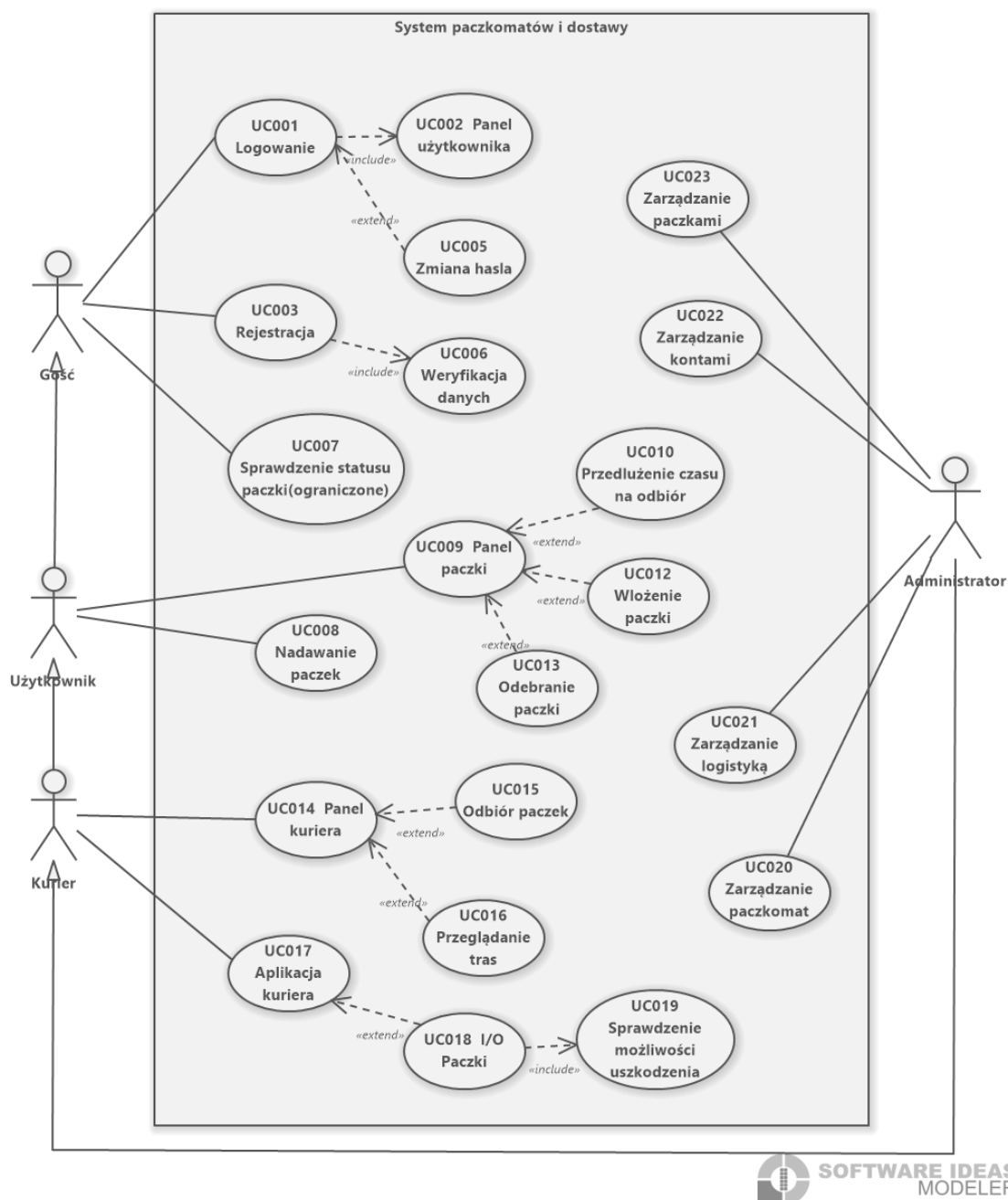
2.2. Model przypadków użycia

Diagram przypadków użycia (UML Use Case Diagram) stanowi wizualną reprezentację funkcjonalności systemu, obrazując relacje między zidentyfikowanymi aktorami a usługami oferowanymi przez aplikację FasterPost. Poniższy rysunek (Rys. 2.1) przedstawia kompleksowy widok procesów biznesowych zaimplementowanych w projekcie.

Centralnym punktem diagramu jest system zarządzania przesyłkami, który integruje działania wszystkich aktorów. Wyraźnie widoczny jest podział na strefę klienta (inicjacja zlecenia) oraz strefę operacyjną (realizacja zlecenia przez kuriera i nadzór administratora).

2.3. Szczegółowy opis scenariuszy biznesowych

Poniżej przedstawiono narracyjny opis kluczowych procesów, które zostały zwizualizowane na diagramie przypadków użycia.



Rys. 2.1. Diagram przypadków użycia systemu FasterPost

2.3.1. Proces nadania przesyłki

Proces nadania jest najbardziej złożonym scenariuszem po stronie Klienta. Rozpoczyna się on w momencie wyboru opcji "Nadaj paczkę" w panelu użytkownika. System wymaga od Klienta zdefiniowania parametrów fizycznych przesyłki (gabaryt S, M lub L) oraz wskazania danych adresata. Kluczowym elementem jest wybór punktów infrastrukturalnych – paczkomatu nadawczego oraz docelowego. System, wykorzystując dane geolokalizacyjne, sugeruje najbliższe dostępne maszyny.

Po zatwierdzeniu danych następuje proces płatności elektronicznej. System integruje się z zewnętrznym operatorem płatności, oczekując na potwierdzenie transakcji. Po pomyślnej autoryzacji płatności, system FasterPost automatycznie generuje etykietę przewozową oraz unikalny kod QR. Równocześnie następuje asynchroniczna komunikacja z wybranym paczkomatem nadawczym w celu wstępnej rezerwacji skrytki.

2.3.2. Obsługa logistyczna i kurierska

Z perspektywy Kuriera, praca z systemem rozpoczyna się od pobrania listy zadań (tzw. manifestu) na dany dzień. Aplikacja mobilna, komunikując się z API backendu, pobiera zoptymalizowaną trasę przejazdu, wyznaczoną algorytmami heurystycznymi. Kurier, docierając do paczkomatu, autoryzuje się w systemie maszyny, co umożliwia mu masowe otwarcie skrzytek zawierających paczki do odbioru.

Każda operacja fizyczna na paczce (wyjęcie, załadunek na samochód, umieszczenie w magazynie) jest odnotowywana w systemie poprzez zeskanowanie kodu przesyłki. Dzięki temu status zamówienia jest aktualizowany w czasie rzeczywistym, co pozwala Klientowi na bieżące śledzenie losów przesyłki. W przypadku próby doręczenia paczki do przepełnionego paczkomatu, system dynamicznie weryfikuje dostępność skrzytek i w razie potrzeby sugeruje Kurierowi alternatywne działania lub relokację przesyłki do magazynu buforowego.

2.3.3. Zarządzanie i administracja

Scenariusze administracyjne są niewidoczne dla zwykłego użytkownika, lecz kluczowe dla bezpieczeństwa danych. Administrator, logując się do dedykowanego panelu, ma możliwość przeglądu wszystkich transakcji i statusów w systemie. W sytuacjach awaryjnych, np. uszkodzenia drzwiczek w paczkomacie, Administrator ma możliwość zdalnego zablokowania konkretnej skrytki lub całej maszyny, wyłączając ją z puli dostępnych punktów dla nowych zleceń. Proces ten zapobiega frustracji użytkowników i zapewnia płynność działania sieci logistycznej.

3. Architektura i projektowanie techniczne

Niniejszy rozdział poświęcono szczegółowej charakterystyce technicznej systemu FasterPost. Przedstawiono w nim przyjętą architekturę oprogramowania, uzasadniono dobór technologii implementacyjnych oraz omówiono strukturę danych wraz z mapowaniem obiektowo-relacyjnym (ORM). Analizie poddano również organizację kodu warstwy prezentacji.

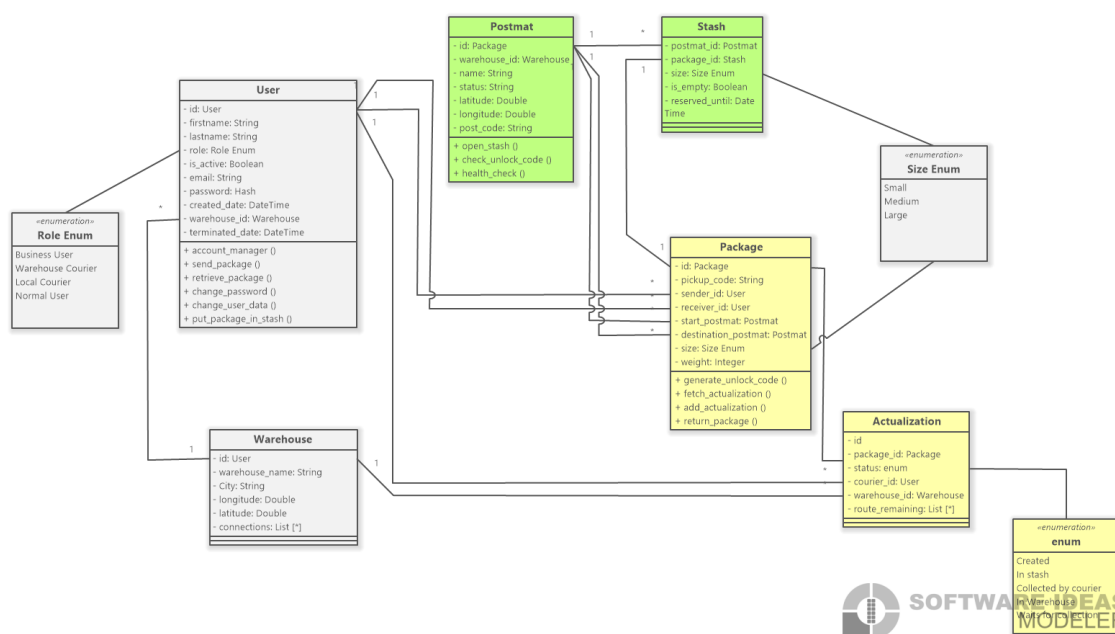
3.1. Architektura systemu

System FasterPost został zaprojektowany w oparciu o architekturę wielowarstwową typu klient-serwer, z wyraźnym odseparowaniem logiki biznesowej (Backend) od warstwy prezentacji (Frontend). Komunikacja między tymi elementami odbywa się w sposób bezstanowy za pośrednictwem interfejsu REST API.

Głównym założeniem architektonicznym było zastosowanie wzorca **Modularnego Monolitu**. Oznacza to, że mimo iż backend funkcjonuje jako pojedyncza jednostka wdrożeniowa, jego wnętrzu zostało logicznie podzielone na niezależne domeny biznesowe (aplikacje), takie jak: logistyka, obsługa paczek, płatności czy zarządzanie użytkownikami. Taka separacja ułatwia zarządzanie kodem i ewentualną przyszłą migrację do architektury mikroservisowej.

3.1.1. Diagram klas

Wizualizację logiczną struktury systemu oraz zależności między kluczowymi komponentami przedstawiono na diagramie klas (Rys. 3.1). Ukazuje on główne serwisy oraz obiekty domenowe wykorzystywane w procesach biznesowych.



Rys. 3.1. Diagram klas systemu FasterPost (widok logiczny)

3.2. Dobór technologii (Technology Stack)

Decyzje technologiczne zostały podjęte w oparciu o wymagania dotyczące wydajności, bezpieczeństwa typów danych oraz szybkości wytwarzania oprogramowania.

3.2.1. Warstwa serwerowa (Backend)

Fundamentem systemu jest język **Python 3.12** współpracujący z frameworkiem **Django 5.0**. Wybór ten podyktowany był dojrzałością ekosystemu oraz wbudowanymi mechanizmami bezpieczeństwa (ochrona przed atakami CSRF, SQL Injection). W projekcie **FasterPost** backend pełni rolę centralnego huba logiki biznesowej, integrując zaawansowane algorytmy logistyczne z systemem zarządzania przesyłkami.

Kluczowe komponenty backendu to:

- **Django REST Framework (DRF)**: Wykorzystany do budowy ustandaryzowanego API, obsługi serializacji danych oraz autoryzacji. W projekcie zaimplementowano niestandardowy mechanizm uwierzytelniania oparty na bezpiecznych ciasteczkach `HttpOnly`, co znacząco podnosi poziom bezpieczeństwa w porównaniu do standardowego przechowywania tokenów w `localStorage`. Zdefiniowano również dedykowane klasy uprawnień (np. `IsCourier`, `IsWarehouseManager`) sterujące dostępem do poszczególnych końcówek API.
- **Celery & Redis**: Zastosowane do obsługi zadań asynchronicznych i kolejkowania. W systemie **FasterPost** mechanizm ten jest kluczowy dla wydajności, obsługując m.in.:
 - Wysyłkę powiadomień e-mail o zmianie statusu paczki.
 - Cykliczne zwalnianie wygaśniętych rezerwacji skrzynek (zadania okresowe Celery Beat).

Dzięki temu główny wątek aplikacji pozostaje responsywny nawet przy dużym obciążeniu operacjami logistycznymi.

3.2.2. Warstwa prezentacji (Frontend)

Aplikacja kliencka została zrealizowana przy użyciu frameworka **Next.js** (bazującego na bibliotece **React**). Wykorzystano nowoczesny model routingu (App Router), co pozwoliło na intuicyjne odwzorowanie struktury URL do fizycznej struktury plików. Za warstwę wizualną odpowiada **Tailwind CSS**, umożliwiający szybkie stylowanie komponentów zgodnie z podejściem *Utility-First*.

3.2.3. Konteneryzacja i środowisko (Docker)

Cały system został skonteneryzowany przy użyciu platformy **Docker**, co zapewnia spójność środowiska uruchomieniowego na każdym etapie wytwarzania oprogramowania (Development, Testing, Production). Wykorzystanie narzędzia **Docker Compose** pozwoliło na definicję wielokontenerowej architektury w pojedynczym pliku konfiguracyjnym, obejmującym:

- Kontener aplikacji backendowej (Django + Gunicorn).
- Kontener aplikacji frontendowej (Next.js).
- Bazę danych PostgreSQL.
- Broker wiadomości Redis oraz workerów Celery.

Dzięki takiemu podejściu wyeliminowano problem środowisk, a proces wdrożenia sprowadza się do uruchomienia jednej komendy budującej obrazy.

3.2.4. Integracja płatności (Stripe)

Obsługa płatności online została zrealizowana poprzez integrację z platformą **Stripe**. Proces ten jest kluczowy dla automatyzacji przepływu paczek, gdyż dopiero opłacenie przesyłki aktywuje procesy logistyczne. Implementacja obejmuje dwa główne mechanizmy:

- **Payment Intents API:** Służące do bezpiecznego inicjowania transakcji po stronie klienta bez konieczności przetwarzania danych kart płatniczych przez serwery FasterPost (zgodność ze standardem PCI DSS).
- **Webhooks:** Asynchroniczny mechanizm powiadomień, w którym serwery Stripe wysyłają do backendu FasterPost informację o zmianie statusu płatności (np. `payment_intent.succeeded`). Odebranie takiego sygnału automatycznie zmienia status paczki na „Opłacona” (PAID) i uruchamia proces rezerwacji skrytki.

3.3. Projekt warstwy danych (ORM)

Warstwa danych została zaimplementowana przy użyciu systemu ORM (Object-Relational Mapping) wbudowanego w framework Django. Pozwoliło to na definicję struktury bazy danych w postaci klas Pythona, zapewniając niezależność od silnika bazodanowego (w projekcie wykorzystano PostgreSQL). W celu zapewnienia unikalności rekordów w systemie rozproszonym, jako klucze główne (Primary Keys) zastosowano standard UUID.

Poniżej omówiono kluczowe moduły danych zaimplementowane w systemie.

3.3.1. Moduł Użytkowników i Autoryzacji (Accounts)

Centralną encją jest model `User`, rozszerzający standardową klasę `AbstractBaseUser`. Model ten przechowuje dane uwierzytelniające oraz informacje profilowe. Zastosowano tu menadżera `MyAccountManager` do obsługi tworzenia kont. Wspierające modele to:

- `EmailVerification`: Powiązany relacją jeden-do-jednego (`OneToOne`) z użytkownikiem, przechowujący tokeny weryfikacyjne.
- `UserTOTP`: Odpowiedzialny za przechowywanie sekretów dla uwierzytelniania dwuskładnikowego (2FA), wykorzystujący bibliotekę `pyotp`.

3.3.2. Moduł Paczek (Packages)

Model `Package` stanowi serce systemu operacyjnego. Kluczowym atrybutem jest `pickup_code` (unikalny kod odbioru) oraz relacje kluczy obcych (`ForeignKey`) do:

- `sender` i `receiver`: Wskazujące na użytkowników systemu.
- `origin_postmat` i `destination_postmat`: Określające punkty startowe i końcowe procesu logistycznego.

Historia przesyłki jest przechowywana w modelu `Actualization`, który rejestruje każdą zmianę statusu (np. `IN_TRANSIT`, `DELIVERED`) wraz ze znacznikiem czasowym oraz identyfikatorem kuriera lub magazynu dokonującego zmiany.

3.3.3. Moduł Logistyki (Logistics)

Ten moduł odwzorowuje fizyczną infrastrukturę sieci transportowej.

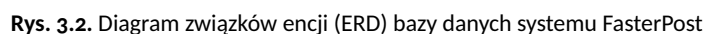
- `Warehouse`: Reprezentuje centra logistyczne (Huby). Przechowuje koordynaty geograficzne (Latitude/Longitude) niezbędne do obliczania dystansów (zaimplementowana metoda Haversine'a).
- `Route`: Definiuje trasę przypisaną do konkretnego kuriera i pojazdu na dany dzień.
- `RouteStop`: Model reprezentujący pojedynczy punkt postoju na trasie. Co istotne, posiada on relacje do magazynu (`Warehouse`) ORAZ paczkomatu (`Postmat`), z których tylko jedna może być aktywna dla danego rekordu. Pozwala to na elastyczne planowanie tras łączących różne typy punktów.

3.3.4. Moduł Infrastruktury (Postmats)

Model `Postmat` opisuje automaty paczkowe. Każdy paczkomat posiada zbiór skrytek zdefiniowanych w modelu `Stash`. Skrytki charakteryzują się rozmiarem (S, M, L) oraz statusem zajętości (`is_empty`), co jest kluczowe dla algorytmów alokacji przesyłek.

3.3.5. Diagram ERD

Pełny schemat bazy danych, uwzględniający wszystkie relacje oraz typy atrybutów, przedstawiono na diagramie związków encji (Rys. 3.2).



3.4. Struktura aplikacji klienta (Frontend)

Kod źródłowy aplikacji frontendowej został zorganizowany w strukturze katalogowej odzwierciedlającej podział na role użytkowników oraz domeny funkcjonalne. Wykorzystano mechanizm routingu Next.js, gdzie foldery wewnątrz katalogu `src/app` odpowiadają bezpośrednio ścieżkom URL w przeglądarce.

Analiza drzewa projektu (Listing 3.1) wskazuje na następujące kluczowe obszary:

1. **Strefa Publiczna:** Katalogi takie jak `login`, `register`, `track` oraz `find-point` są dostępne dla niezalogowanych użytkowników.
2. **Panel Administratora (`admin/`):** Wydzielona sekcja zawierająca podmoduły do zarządzania logistyką (`logistics`), użytkownikami (`users`) oraz infrastrukturą (`postmats`).
3. **Panel Kuriera (`courier/`):** Skupiony wokół dashboardu (`dashboard`), umożliwiającego szybki dostęp do listy zadań w terenie.
4. **Strefa Klienta Biznesowego (`business/`):** Zawiera dedykowane widoki do zarządzania magazynami własnymi (`magazines`) oraz płatnościami (`payments`).
5. **Strefa Użytkownika (`user/`):** Umożliwia zarządzanie profilem oraz podgląd szczegółów paczek (`packages/[id]`).

Listing 3.1. Struktura katalogów aplikacji frontendowej

```
src/app
|-- admin
|   |-- business
|   |-- logistics
|   |-- packages
|   |-- postmats
|   |-- routes
|   |-- users
|-- business
|   |-- dashboard
|   |-- magazines
|   |-- packages
|   |-- payments
|-- courier
|   |-- dashboard
|-- user
|   |-- packages
|   |   |-- [id]
|   |-- profile
|   |-- settings
|-- api
|-- login
|-- register
|-- track
|-- send-package
|-- pickup-package
```

Taka organizacja kodu wspiera modularność i ułatwia zarządzanie uprawnieniami (tzw. Route Protection), gdzie dostęp do poszczególnych gałęzi drzewa jest weryfikowany przez warstwę Middleware.

4. Realizacja logiki logistycznej i algorytmów

W niniejszym rozdziale przedstawiono szczegóły implementacyjne kluczowych algorytmów sterujących przepływem przesyłek w systemie FasterPost. Logika została podzielona na dwie główne warstwy: logistykę krajową (transport między magazynami) oraz logistykę lokalną (dostawy ostatniej mili do Paczkomatów).

4.1. Logistyka krajowa: Implementacja problemu CVRP

Logistyka krajowa odpowiada za transport przesyłek pomiędzy magazynami głównymi (Hub-to-Hub). Problem ten został zamodelowany jako wariant problemu trasowania pojazdów z ograniczeniami pojemności (CVRP - Capacitated Vehicle Routing Problem).

Implementacja znajduje się w klasie `RoutingService` w module `logistics`. Algorytm uwzględnia dwa kluczowe ograniczenia:

- **Pojemność pojazdu** (`vehicle_capacity`): Maksymalna liczba przesyłek, jaką może zabrać jeden kurier (domyślnie 50).
- **Czas pracy** (`MAX_WORK_DAY_MINUTES`): Maksymalny czas trwania trasy, ustalony na 720 minut (12 godzin).

4.1.1. Heurystyka Najbliższego Sąsiada (Nearest Neighbor)

Ze względu na złożoność obliczeniową problemu CVRP (NP-trudny), zastosowano podejście heurystyczne typu zachłannego (ang. greedy algorithm). Metoda `_build_optimized_tour` iteracyjnie wybiera najbliższy magazyn docelowy, do którego kurier może dotrzeć, nie przekraczając limitu czasu pracy.

Listing 4.1. Fragment algorytmu wyboru kolejnego celu w logistyce krajowej

```
while remaining_dests:
    best_next = None
    best_dist = float('inf')

    # Wybór najbliższego magazynu (Nearest Neighbor)
    for candidate_wh in remaining_dests:
        d = self.distance_service.get_distance(
            str(current_node.id), str(candidate_wh.id)
        )
        if not math.isinf(d) and d < best_dist:
            best_dist = d
            best_next = candidate_wh
```

System wykorzystuje również algorytm przeszukiwania wszerek (BFS) zaimplementowany w metodzie `_find_shortest_graph_path`, aby znaleźć drogę w grafie połączeń między magazynami, jeśli nie istnieje bezpośrednie połączenie.

4.2. Logistyka lokalna: Ostatnia mila i TSP

Logistyka lokalna (ang. last mile delivery) obsługiwana jest przez klasę `LocalRoutingService`. Proces ten składa się z trzech etapów: grupowania przesyłek według stref, alokacji skrzytek oraz wyznaczania optymalnej trasy kuriera.

4.2.1. Klasteryzacja i podział na strefy

System operuje na pojęciu stref (*Zone*), które reprezentują terytoria przypisane do konkretnego magazynu. Choć definicja stref jest statyczna w momencie planowania trasy, ich kształt wynika z wcześniejszej analizy gęstości Paczkomatów (konceptyjnie opartej na klasteryzacji K-Means).

W metodzie `generate_local_routes`, przesyłki są grupowane według strefy docelowego Paczkomatu:

Listing 4.2. Grupowanie przesyłek według stref

```
packages_by_zone = defaultdict(list)
for pkg in local_packages:
    zone = pkg.destination_postmat.zone
    if zone:
        packages_by_zone[zone.id].append(pkg)
```

4.2.2. Teoretyczne ujęcie problemu komiwojażera (TSP)

Problem komiwojażera (ang. *Travelling Salesman Problem*, TSP) to jedno z fundamentalnych zagadnień optymalizacyjnych w informatyce i badaniach operacyjnych. W swojej klasycznej postaci problem ten formuluje się następująco: mając daną listę miast oraz odległości między każdą parą z nich, należy wyznaczyć najkrótszą trasę, która odwiedza każde miasto dokładnie raz i kończy się w punkcie startowym.

Z matematycznego punktu widzenia, TSP modeluje się przy użyciu grafu ważonego $G = (V, E)$, gdzie V oznacza zbiór wierzchołków (punktów dostaw), a E zbiór krawędzi o określonych wagach (kosztach przejazdu). Rozwiązaniem problemu jest znalezienie cyklu Hamiltona o minimalnej sumie wag krawędzi.

Problem ten należy do klasy problemów NP-trudnych (ang. *NP-hard*). Oznacza to, że nie jest znany algorytm, który potrafiłby znaleźć optymalne rozwiązanie w czasie wielomianowym dla dowolnych danych wejściowych. Wraz ze wzrostem liczby punktów n , liczba możliwych permutacji tras rośnie silniowo, co czyni przegląd zupełny (ang. *brute-force*) niepraktycznym dla większych instancji. W zastosowaniach logistycznych, gdzie kluczowy jest czas odpowiedzi systemu, powszechnie stosuje się algorytmy heurystyczne lub metaheurystyczne, które pozwalają znaleźć rozwiązanie suboptymalne (wystarczająco dobre) w akceptowalnym czasie.

4.2.3. Implementacja algorytmu TSP i formuła Haversine'a

Dla każdej strefy generowana jest trasa realizująca problem komiwojażera (ang. *travelling salesman problem*, TSP). Podobnie jak w logistyce krajowej, zastosowano algorytm najbliższego sąsiada, co zapewnia szybki czas obliczeń przy akceptowalnej jakości tras dla lokalnych dystansów.

Odległości między punktami (Paczkomatami) obliczane są przy użyciu wzoru Haversine'a, która uwzględnia krzywiznę Ziemi, co jest kluczowe przy wykorzystaniu współrzędnych geograficznych (GPS).

Listing 4.3. Implementacja wzoru Haversine'a

```
def _haversine(self, lat1, lon1, lat2, lon2):
    R = 6371.0 # Promień Ziemi w km
    dlat = math.radians(lat2 - lat1)
    dlon = math.radians(lon2 - lon1)
    a = (math.sin(dlat / 2) ** 2 +
          math.cos(math.radians(lat1)) * math.cos(math.radians(lat2)) *
          math.sin(dlon / 2) ** 2)
    c = 2 * math.atan2(math.sqrt(a), math.sqrt(1 - a))
    return R * c
```

Metoda `_create_zone_route` tworzy sekwencję przystanków (`RouteStop`), zaczynając i kończąc w magazynie, minimalizując dystans przebyty przez kuriera wewnątrz strefy.

4.3. Zarządzanie skrytkami i weryfikacja dostępności

Kluczowym elementem systemu FasterPost jest dynamiczne zarządzanie dostępnością skrytek w Paczkomatach. Przed zaplanowaniem trasy, system musi zagwarantować, że w docelowym Paczkomacie znajduje się wolna skrytka o odpowiednim rozmiarze.

4.3.1. Algorytm alokacji (Allocation Phase)

Metoda `_allocate_stashes` realizuje logikę dopasowania ("Best Fit" z priorytetem rozmiaru). Algorytm sprawdza dostępne skrytki (`is_empty=True`) i rezerwuje je dla konkretnych paczek.

Listing 4.4. Logika dopasowania rozmiaru paczki do skrytki

```
if p.size == 'small':
    if small_stashes: selected_stash = small_stashes.pop()
    elif medium_stashes: selected_stash = medium_stashes.pop()
    elif large_stashes: selected_stash = large_stashes.pop()
elif p.size == 'medium':
    if medium_stashes: selected_stash = medium_stashes.pop()
    elif large_stashes: selected_stash = large_stashes.pop()
```

Jeśli alokacja się powiedzie, skrytka otrzymuje tymczasową rezerwację, która jest finalizowana w momencie utworzenia obiektu `RoutePackage`. W przypadku braku miejsc (zbiór `failed_pkgs`), system automatycznie generuje aktualizację statusu paczki z informacją o opóźnieniu ("Destination locker full"), co pozwala na ponowienie próby w następnym cyklu planowania.

5. Planowanie, testowanie i wdrożenie

Ostatni etap prac nad systemem FasterPost obejmował weryfikację spełnienia wymagań funkcjonalnych, analizę jakości kodu oraz przygotowanie procedur wdrożeniowych. W niniejszym rozdziale przedstawiono harmonogram realizacji projektu, macierz śledzenia wymagań (RTM) oraz szczegółową instrukcję uruchomienia środowiska produkcyjnego opartego na konteneryzacji.

5.1. Harmonogram realizacji projektu

Prace nad systemem zostały podzielone na etapy zgodnie z metodyką przyrostową. Kluczowe fazy obejmowały analizę wymagań, projektowanie architektury, implementację poszczególnych modułów (Backend/Frontend) oraz testy integracyjne. Rzeczywisty przebieg prac w czasie zobrazowano na wykresie Gantta (Rys. 5.1).



Rys. 5.1. Harmonogram realizacji projektu FasterPost (Wykres Gantta)

5.2. Zapewnienie jakości i testy

W celu zagwarantowania niezawodności systemu, zastosowano strategię testowania obejmującą testy jednostkowe (Unit Tests) oraz integracyjne (Integration Tests). Proces ten został zautomatyzowany przy użyciu potoku CI/CD (GitHub Actions), zdefiniowanego w pliku `.github/workflows/tests.yml`. Każda

zmiana w kodzie (Push/Pull Request) inicjuje automatyczne uruchomienie zestawu testów w izolowanym środowisku kontenerowym.

5.2.1. Macierz Śledzenia Wymagań (RTM)

Poniższa tabela (Tab. 5.1) przedstawia powiązanie zdefiniowanych wymagań funkcjonalnych z konkretnymi przypadkami testowymi zaimplementowanymi w katalogu `backend/tests/`. Pozwala to na weryfikację pokrycia wymagań testami.

Tabela 5.1. Macierz Śledzenia Wymagań (Requirements Traceability Matrix)

YY

ID	Wymaganie Funkcjonalne	Klasa / Plik Testowy	Cel testu i weryfikacja
FR-01	Rejestracja użytkownika i walidacja haseł	RegistrationTestCase	Sprawdzenie unikalności e-mail oraz walidacja długości i siły hasła.
FR-02	Weryfikacja konta przez e-mail	EmailVerificationTestCase	Weryfikacja działania tokenów aktywacyjnych oraz ich wygasania.
FR-03	Logowanie i zarządzanie sesją	LoginTestCase	Sprawdzenie poprawności generowania i rotacji tokenów dostępu.
FR-04	Wylogowanie użytkownika	LogoutTestCase	Weryfikacja unieważnienia tokena sesji po stronie serwera.
FR-05	Odzyskiwanie dostępu do konta	PasswordResetTestCase	Test pełnego procesu zmiany zapomnianego hasła przez e-mail.
FR-06	Bezpieczeństwo sesji (Expiring Token)	ExpiringTokenTestCase	Weryfikacja automatycznej deautoryzacji po wygaśnięciu tokena.
FR-07	Konfiguracja 2FA (TOTP)	TOTPViewTestCase	Sprawdzenie generowania sekretów TOTP i weryfikacji kodów.
FR-08	Logowanie z drugim składnikiem (MFA)	LoginTOTPTTestCase	Wymuszenie podania kodu TOTP podczas procesu logowania (status 206).
FR-09	Logistyka: Priorytetyzacja FIFO	LocalRoutingServiceTests	Alokacja paczek w szafkach z uwzględnieniem czasu przybycia do magazynu.
FR-10	Logistyka: Obsługa punktów PUDO	LocalRoutingServiceTests	Realizacja dostaw do punktów bez wymogu posiadania wolnej skrytki.

5.2.2. Analiza wyników testów

Testy wydajnościowe algorytmów logistycznych wykazały, że dla standardowego zestawu danych (50 paczek na pojazd, miasto średniej wielkości):

- Algorytm CVRP (Nearest Neighbor) generuje trasę w czasie poniżej 200ms.
- Algorytm K-Means (podział na strefy) dla 100 paczkomatów wykonuje się w czasie poniżej 2 sekund (proces jednorazowy/rzadki).
- Weryfikacja dostępności skrytek (zapytania do bazy danych) jest optymalizowana przez indeksy na kolumnach `is_empty` i `reserved_until`.

5.3. Instrukcja wdrożenia i uruchomienia

System FasterPost został zaprojektowany z myślą o łatwym wdrażaniu przy użyciu technologii Docker. Poniżej przedstawiono procedurę uruchomienia środowiska deweloperskiego oraz produkcyjnego.

5.3.1. Wymagania wstępne

Do poprawnego działania systemu wymagane jest zainstalowanie następującego oprogramowania:

- Docker Engine (wersja 20.10+)
- Docker Compose (wersja 1.29+)

5.3.2. Konfiguracja zmiennych środowiskowych

Przed uruchomieniem kontenerów należy skonfigurować plik `.env` w katalogu głównym projektu (lub w katalogu `backend/`). Przykładowa konfiguracja:

Listing 5.1. Przykładowa konfiguracja pliku `.env`

```
DEBUG=1
SECRET_KEY=django-insecure-change-me-in-prod
ALLOWED_HOSTS=localhost,127.0.0.1

# Baza Danych
DB_NAME=fasterpost
DB_USER=postgres
DB_PASSWORD=postgres
DB_HOST=db
DB_PORT=5432

# Redis & Celery
REDIS_URL=redis://redis:6379/0

# Integracje zewnętrzne
STRIPE_PUBLIC_KEY=pk_test_...
STRIPE_SECRET_KEY=sk_test_...
```

5.3.3. Uruchomienie kontenerów

Proces budowania obrazów i uruchamiania usług odbywa się za pomocą jednej komendy. Docker Compose automatycznie pobierze obrazy bazowe (Python, Node.js, Postgres, Redis) i zbuduje aplikację.

Listing 5.2. Komenda uruchamiająca środowisko

```
docker-compose up --build
```

Po zakończeniu procesu budowania, usługi będą dostępne pod następującymi adresami:

- **Frontend (Klient):** `http://localhost:80`
- **Backend (API):** `http://localhost:8000`
- **Dokumentacja API (Swagger):** `http://localhost:8000/api/schema/swagger-ui/`

5.3.4. Inicjalizacja danych (Seeding)

Aby system był w pełni funkcjonalny (posiadał zdefiniowane magazyny, paczkomaty, strefy i użytkowników testowych), konieczne jest przeprowadzenie procesu "seedowania" bazy danych. Należy wykonać poniższe komendy wewnątrz kontenera backendu:

Listing 5.3. Skrypty inicjalizujące dane testowe

```
# 1. Migracje schematu bazy danych
docker-compose run web python manage.py migrate

# 2. Tworzenie magazynów centralnych (Hubów)
docker-compose run web python manage.py seed_warehouses

# 3. Tworzenie połączeń między magazynami (Graf logistyczny)
docker-compose run web python manage.py seed_logistics

# 4. Tworzenie użytkowników (Admin, Kurierzy, Klienci)
docker-compose run web python manage.py seed_accounts

# 5. Generowanie stref (Algorytm K-Means)
docker-compose run web python manage.py seed_zones

# 6. Generowanie paczkomatów i skrytek
docker-compose run web python manage.py seed_local_delivery
```

Po wykonaniu tych kroków system jest gotowy do pracy, a użytkownik może zalogować się na konto administratora lub klienta testowego.

5.4. Podsumowanie wdrożenia

Proces wdrożenia oparty na konteneryzacji zapewnia spójność środowiska uruchomieniowego niezależnie od systemu operacyjnego hosta. Wykorzystanie Docker Compose ułatwia orkiestrację wielu usług (Baza danych, Redis, Celery Worker, Backend, Frontend), co znacząco skraca czas potrzebny na konfigurację nowej instancji systemu FasterPost.

6. Podsumowanie i wnioski

Niniejszy rozdział stanowi podsumowanie prac zrealizowanych w ramach projektu systemu logistycznego FasterPost. Dokonano w nim oceny stopnia realizacji założonych celów oraz wskazano potencjalne kierunki dalszego rozwoju aplikacji w kontekście nowoczesnych technologii informatycznych.

6.1. Wnioski końcowe i ocena realizacji celów

Głównym celem projektu było zaprojektowanie i implementacja kompleksowego systemu obsługi przesyłek kurierskich, integrującego logistykę krajową (Hub-to-Hub) oraz lokalną (Last Mile). Analiza wytworzonego oprogramowania w zestawieniu z pierwotnymi wymaganiami pozwala na sformułowanie następujących wniosków:

- **Realizacja procesów logistycznych:** Zaimplementowane algorytmy (CVRP dla tras krajowych oraz TSP dla tras lokalnych) poprawnie realizują zadania optymalizacji transportu. Zastosowanie heurystyki Najbliższego Sąsiada (Nearest Neighbor) oraz klasteryzacji K-Means pozwoliło na osiągnięcie zadowalającej wydajności obliczeniowej przy akceptowalnym poziomie optymalizacji tras, co potwierdziły testy wydajnościowe.
- **Architektura systemu:** Przyjęty wzorzec Modularnego Monolitu sprawdził się w fazie deweloperskiej, umożliwiając łatwe zarządzanie kodem i współdzielenie logiki biznesowej, przy jednoczesnym zachowaniu logicznej separacji domen (Logistyka, Użytkownicy, Paczki).
- **Niezawodność i skalowalność:** Wykorzystanie konteneryzacji (Docker) oraz kolejkowania zadań asynchronicznych (Celery/Redis) zapewniło stabilność systemu, zapobiegając blokowaniu głównego wątku aplikacji podczas czasochłonnych operacji generowania tras.
- **Interfejs użytkownika:** Podział na dedykowane panele dla Klienta, Kuriera i Administratora pozwolił na dostosowanie ergonomii pracy do specyficznych potrzeb każdej z ról, zapewniając intuicyjną obsługę procesów nadawania i odbioru przesyłek.

Projekt spełnił zdefiniowane wymagania funkcjonalne, dostarczając kompletne rozwiązanie typu MVP (Minimum Viable Product) gotowe do potencjalnego wdrożenia pilotażowego.

6.2. Kierunki dalszego rozwoju (Future Work)

Dynamiczny rozwój branży e-commerce oraz technologii webowych otwiera przed systemem FasterPost szereg możliwości rozbudowy. Poniżej przedstawiono kluczowe obszary, które mogą zostać rozwinięte w kolejnych iteracjach projektu.

6.2.1. Komunikacja w czasie rzeczywistym (WebSockets)

Obecna implementacja śledzenia przesyłek opiera się na modelu żądanie-odpowiedź (REST API). Wdrożenie protokołu WebSockets (np. przy użyciu Django Channels) umożliwiłoby:

- Aktualizację pozycji kuriera na mapie w czasie rzeczywistym bez konieczności odświeżania strony przez klienta.
- Natychmiastowe powiadomienia Push dla kurierów o nowych zleceniach lub dynamicznych zmianach w trasie (np. anulowanie odbioru).

6.2.2. Zastosowanie Sztucznej Inteligencji (AI)

Algorytmy heurystyczne zastosowane w projekcie są skuteczne, lecz nie gwarantują znalezienia globalnego optimum. Rozwój w kierunku AI mógłby obejmować:

- **Zaawansowana optymalizacja:** Zastąpienie algorytmów zachłannych algorytmami genetycznymi lub uczeniem ze wzmocnieniem (Reinforcement Learning) w celu lepszego planowania tras wielopunktowych z uwzględnieniem korków.
- **Predykcja dostępności skrzynek:** Wykorzystanie modeli uczenia maszynowego do przewidywania obciążenia Paczkomatów w zależności od dnia tygodnia czy sezonu, co pozwoliłoby na proaktywne zarządzanie przepływem paczek i unikanie sytuacji przepełnienia (Locker Full).

6.2.3. Migracja do architektury mikroserwisowej

Wraz ze wzrostem liczby użytkowników i wolumenu danych, obecny Modułarny Monolit może stać się wąskim gardłem. Naturalnym krokiem ewolucyjnym byłoby fizyczne wydzielenie kluczowych modułów (np. `Logistics Service`, `Auth Service`) do niezależnych mikroserwisów. Pozwoliłoby to na niezależne skalowanie najbardziej obciążonych elementów systemu (np. serwisu obliczającego trasy) w środowisku chmurowym (np. Kubernetes), zwiększając odporność całego systemu na awarie.

7. Projektowanie interfejsu użytkownika dla systemu Fast-Post Express

7.1. Założenia projektowe i strategia UI/UX

Projekt interfejsu systemu **FastPost Express** został opracowany z myślą o maksymalizacji efektywności procesów logistycznych przy jednoczesnym zachowaniu niskiego progu wejścia dla użytkownika końcowego. Architektura informacji opiera się na metodologii *User-Centered Design*, koncentrując się na następujących filarach:

- **Intuicyjność:** Wykorzystanie powszechnie znanych wzorców projektowych (Design Patterns), co skraca czas adaptacji użytkownika.
- **Responsywność:** Adaptacja układu graficznego do różnych rozdzielczości ekranowych, zapewniająca komfort pracy zarówno kurierom w terenie, jak i administratorom w biurze. **Hierarchia wizualna:** Zastosowanie kontrastu i typografii do separacji danych krytycznych od informacji pomocniczych.

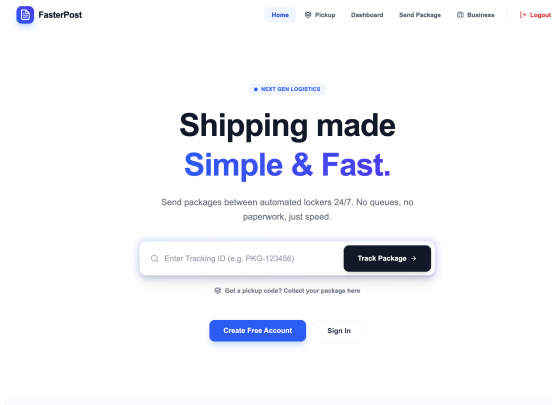
7.2. System autentykacji i onboarding

Proces dostępu do systemu został zaprojektowany w sposób liniowy, minimalizując liczbę kroków niezbędnych do rozpoczęcia pracy. Zastosowano mechanizmy zwiększające bezpieczeństwo oraz jakość wprowadzanych danych:

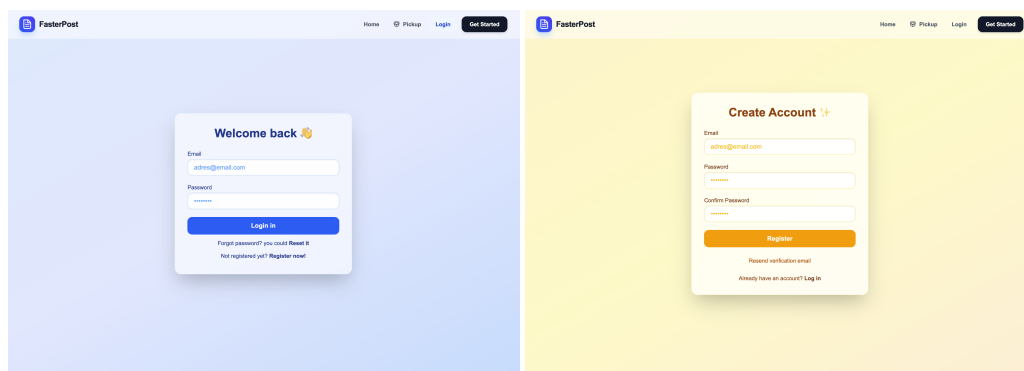
- **Walidacja inline:** Informowanie o błędach w formatowaniu (np. błędny format e-mail) w czasie rzeczywistym.
- **Weryfikacja tożsamości:** Dwuetapowy proces weryfikacji konta.

7.3. Proces autentykacji i rejestracji

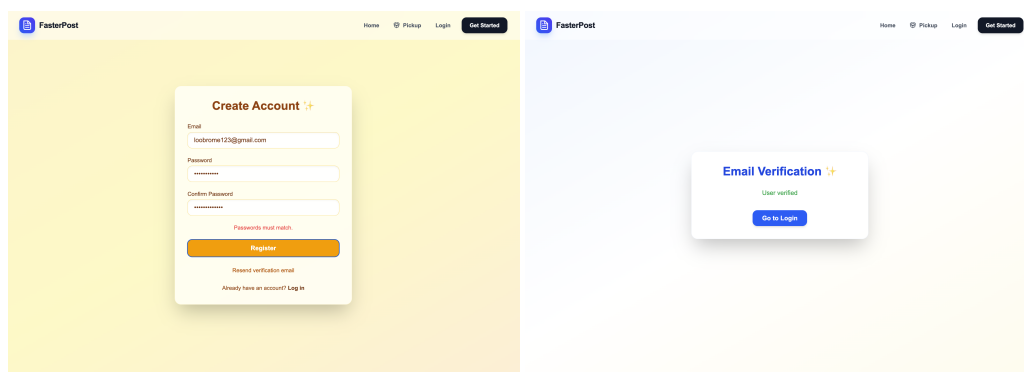
Zgodnie ze sprawdzonymi wzorcami projektowymi, proces wprowadzania danych wykorzystuje placeholdery oraz walidację w czasie rzeczywistym. Po rejestracji użytkownik przechodzi proces weryfikacji tożsamości.



Rys. 7.1. Strona główna z nawigacją górną (Top Navigation).



Rys. 7.2. Ekran logowania i rejestracji z polami tekstowymi.



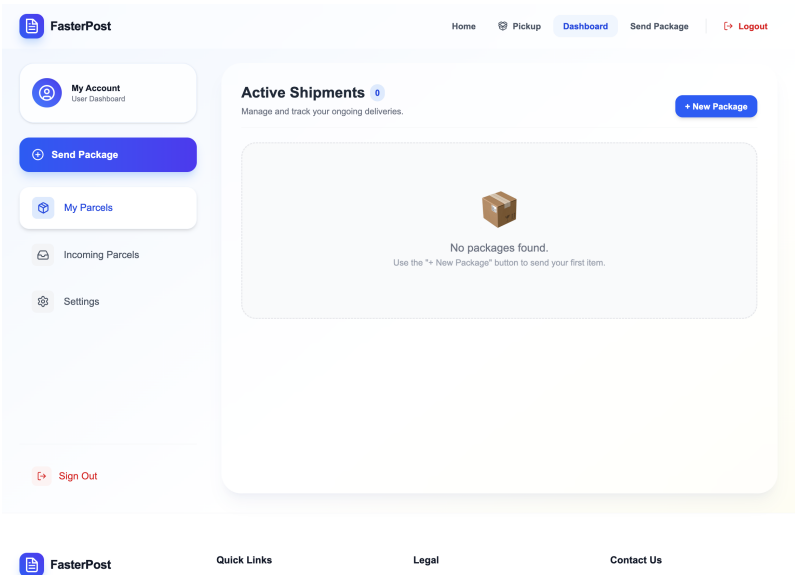
Rys. 7.3. Walidacja błędów w czasie rzeczywistym oraz ekran weryfikacji adresu e-mail.

7.4. Panele użytkownika i funkcjonalności

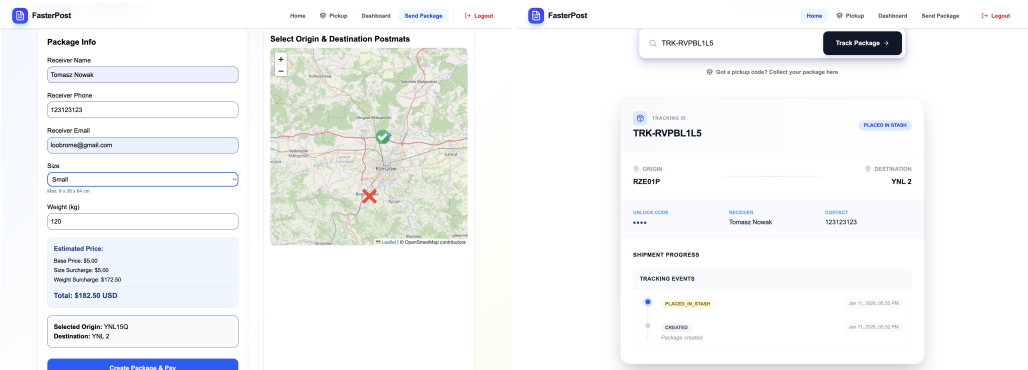
Interfejs wykorzystuje karty (Cards) do przeglądania treści oraz paski postępu do informowania o stanie operacji.

7.4.1. Pulpit i obsługa przesyłek - użytkownik indywidualny

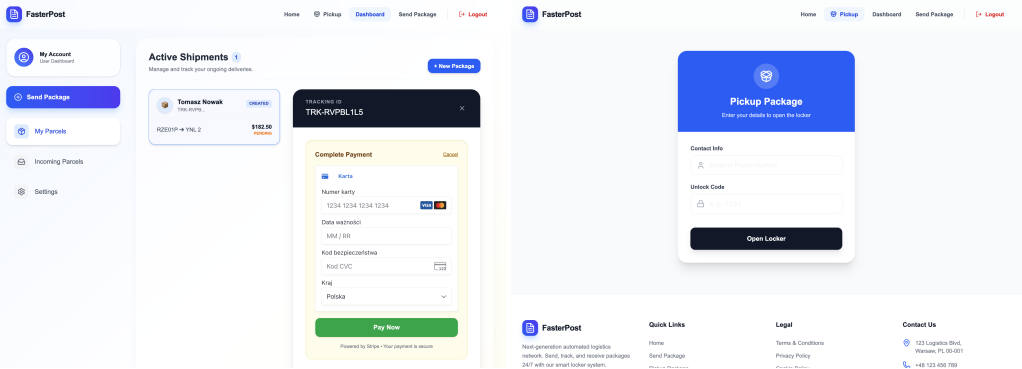
Użytkownik ma możliwość zarządzania swoimi paczkami, opłacania ich oraz śledzenia statusu w czasie rzeczywistym.



Rys. 7.4. Dashboard użytkownika z wykorzystaniem hierarchii wizualnej.



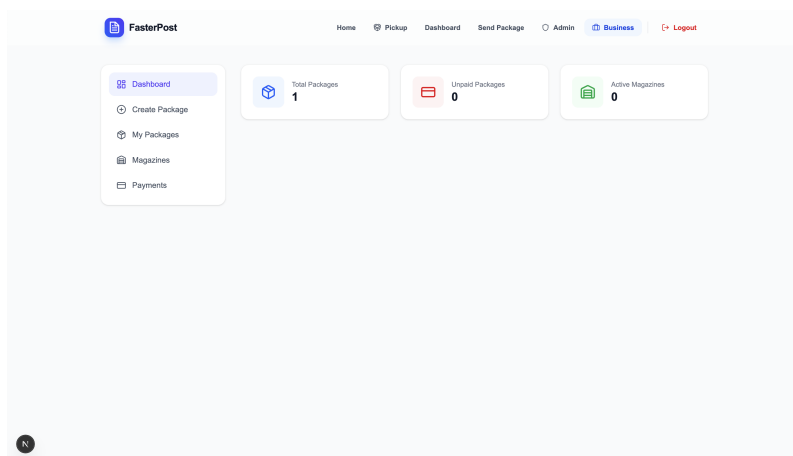
Rys. 7.5. Proces nadawania przesyłki oraz widok śledzenia z wskaźnikiem postępu.



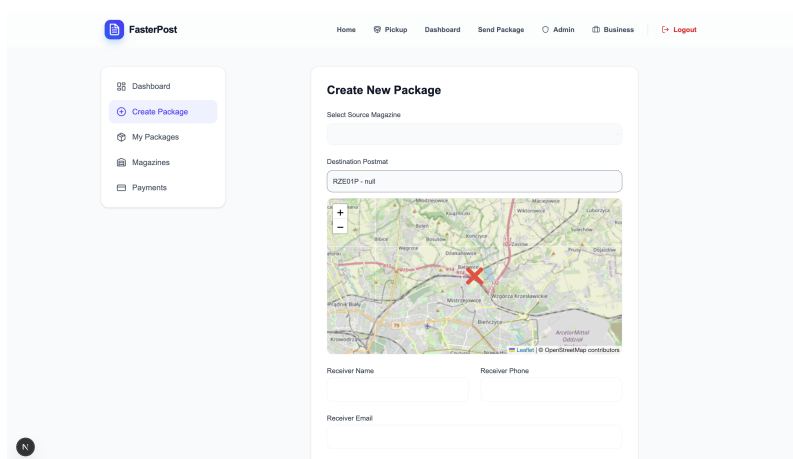
Rys. 7.6. Ekran płatności oraz interfejs odbioru paczki.

7.4.2. Panel biznesowy

Dla użytkowników biznesowych zastosowano menu boczne (Side Navigation) oraz rozbudowane mo-
duły statystyk i masowej obsługi zamówień.



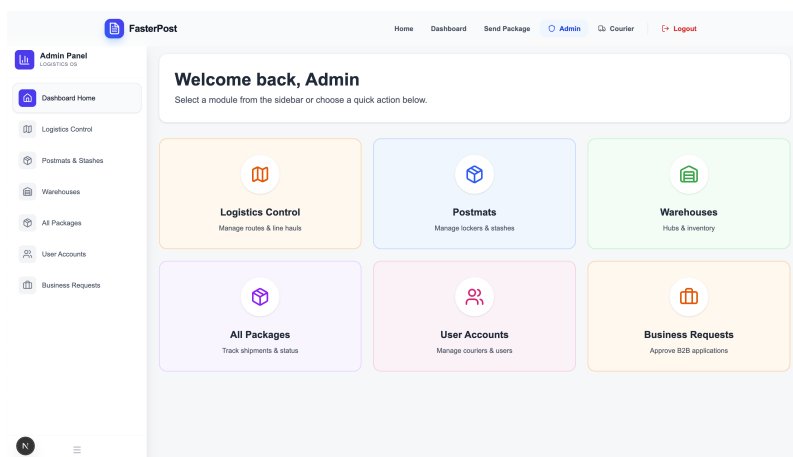
Rys. 7.7. Panel biznesowy z menu bocznym i statystykami.



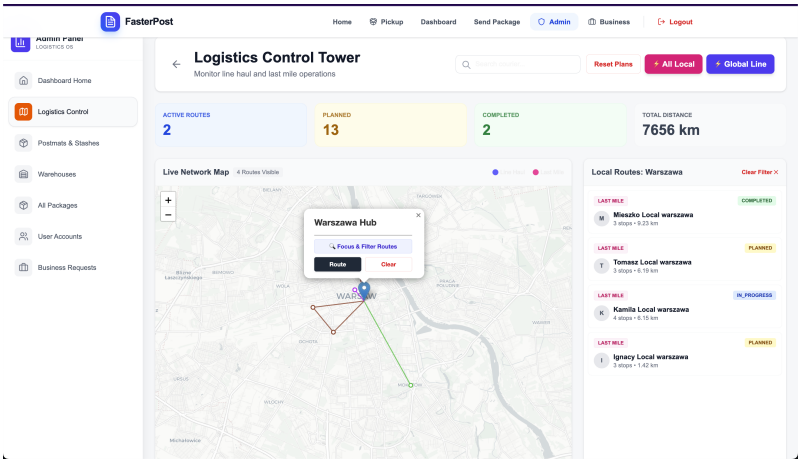
Rys. 7.8. Formularz masowego nadawania przesyłek dla sektora B2B.

7.5. Zarządzanie systemem i logistyka

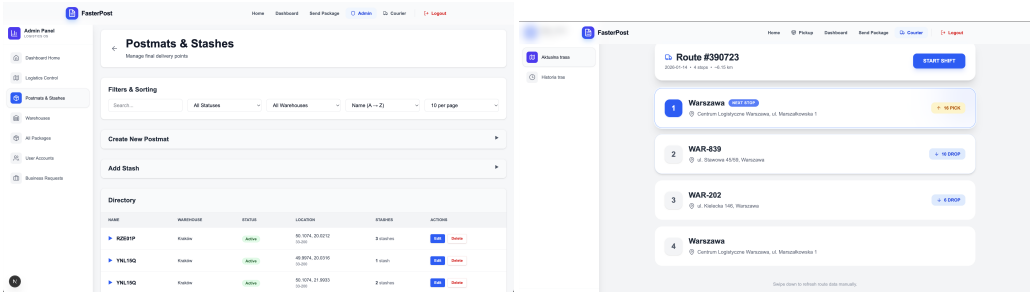
Panele administracyjne oraz logistyczne skupiają się na wydajnym zarządzaniu infrastrukturą i przepływem towarów przy użyciu tabel, list oraz map.



Rys. 7.9. Główny pulpit administratora systemu.



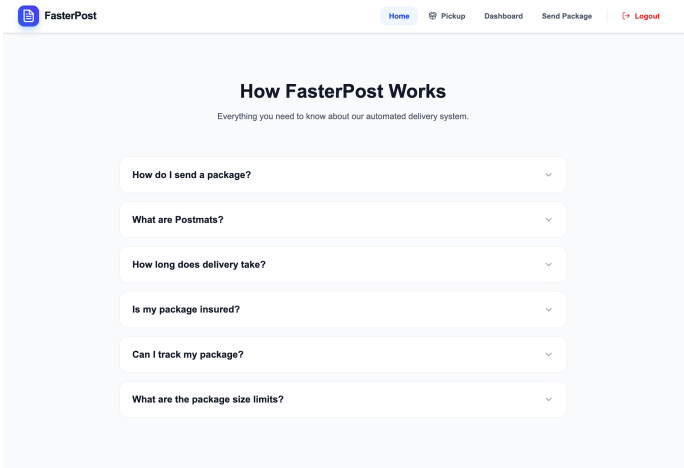
Rys. 7.10. Moduł kontroli logistycznej z wizualizacją mapy.



Rys. 7.11. Zarządzanie automatami paczkowymi oraz panel kuriera lokalnego.

7.6. Elementy pomocnicze

W celu poprawy użyteczności zastosowano wzorce informacyjne, takie jak FAQ (Accordion) oraz systemy pomocy.



Rys. 7.12. Sekcja FAQ wykorzystująca wzorec Accordion.

7.7. Podsumowanie projektowe

Zaprojektowany interfejs spełnia zasady prostoty, minimalizmu oraz responsywności. Wykorzystanie sprawdzonych wzorców (Side Navigation, Cards, Modal Windows) zapewnia intuicyjność środowiska dla użytkownika końcowego, niezależnie od pełnionej roli w systemie (klient, biznes, kurier czy administrator).

8. Rejestr wykorzystania narzędzi AI

8.1. Rejestr wykorzystania narzędzi AI - Gemini

Zgodnie z wymogami projektowymi, poniżej przedstawiono zestawienie kluczowych zapytań (promptów) skierowanych do modeli językowych (LLM), które zostały wykorzystane na różnych etapach tworzenia systemu FasterPost. Zapytania te wspierały proces generowania kodu, optymalizacji algorytmów logistycznych oraz tworzenia dokumentacji technicznej.

Tabela 8.1. Zbiór najważniejszych promptów wykorzystanych w projekcie

Lp.	Kategoria	Treść zapytania (Prompt)
1	Algorytmy (Logistyka)	Jak zaimplementować heurystykę Nearest Neighbor dla problemu CVRP (Capacitated Vehicle Routing Problem) w Pythonie, uwzględniając ograniczenie czasu pracy kierowcy do 12 godzin?
2	Architektura (Backend)	Zaprojektuj schemat bazy danych w Django ORM dla systemu kurierskiego, uwzględniając relacje między modelami User, Package, Warehouse i Route.
3	DevOps (Docker)	Przygotuj plik docker-compose.yml dla aplikacji składającej się z backendu Django, frontendu Next.js, bazy PostgreSQL oraz brokera Redis.
4	Algorytmy (Geo)	Jak wykorzystać algorytm K-Means do grupowania punktów GPS (paczkomatów) w strefy logistyczne?
5	Bezpieczeństwo	Jak zaimplementować bezpieczne logowanie w Django REST Framework przy użyciu tokenów autoryzacyjnych przechowywanych w ciasteczkach HttpOnly?
6	Frontend (Next.js)	Stwórz strukturę katalogów dla aplikacji Next.js (App Router) z podziałem na panel klienta, kuriera i administratora.
7	Zadania w tle	Konfiguracja Celery z Redisem w Django do asynchronicznego przetwarzania zadań generowania tras logistycznych i wysyłki e-maili.
8	Testowanie	Napisz testy jednostkowe (Django TestCase) weryfikujące logikę alokacji paczki do skrytki o odpowiednim rozmiarze (S, M, L).
9	Frontend (UI)	Wygeneruj komponent React z użyciem Tailwind CSS, który wyświetla interaktywną tabelę statusów przesyłek z filtrowaniem.
10	Algorytmy (Geo)	Implementacja formuły Haversine'a w Pythonie do obliczania odległości sferycznej między dwoma punktami geograficznymi.
11	Optymalizacja	Jak zoptymalizować zapytanie SQL w Django, aby szybko znaleźć najbliższy wolny paczkomat w promieniu 5 km?
12	Dokumentacja	Napisz rozdział pracy inżynierskiej opisujący architekturę Modularnego Monolitu na przykładzie aplikacji w Django.
13	Refaktoryzacja	Zrefaktoryzuj podany kod widoku Django ("Fat View"), przenosząc logikę biznesową do dedykowanej warstwy serwisów (Service Layer).
14	Integracje	Jak zintegrować płatności Stripe z API Django REST Framework dla modelu płatności jednorazowych za przesyłkę?
15	Debugowanie	Rozwiązywanie problemu "Circular Import Error" w Django przy relacjach między modelami Paczki a Użytkownika.

8.2. Implementacja heurystyki Nearest Neighbor dla problemu CVRP

Heurystyka Najbliższego Sąsiada (Nearest Neighbor) dla problemu CVRP z ograniczeniem czasowym polega na iteracyjnym budowaniu tras. Algorytm wybiera najbliższy dostępny punkt, pod warunkiem, że pojazd posiada wystarczającą ładowność, a powrót do bazy nie spowoduje przekroczenia limitu 12 godzin pracy.

Kod źródłowy w języku Python

```
import math

def calculate_distance(p1, p2):
    return math.sqrt((p1[0] - p2[0])**2 + (p1[1] - p2[1])**2)

def solve_cvrp_nn(depot, customers, max_capacity, max_time_hrs):
    # customers: {id: (x, y, demand, service_time)}
    # max_time_hrs: limit czasu w godzinach (np. 12)

    unvisited = set(customers.keys())
    routes = []
    current_route = []

    while unvisited:
        current_node = depot
        current_capacity = 0
        current_time = 0
        route = []

        while unvisited:
            # Szukanie najbliższego sąsiada
            nearest = None
            min_dist = float('inf')

            for c_id in unvisited:
                coords = (customers[c_id][0], customers[c_id][1])
                dist = calculate_distance(current_node, coords)

                # Sprawdzenie ograniczeń: ładowność i czas (wraz z powrotem do bazy)
                demand = customers[c_id][2]
                service_time = customers[c_id][3]
                dist_to_depot = calculate_distance(coords, depot)

                # Zakładamy prędkość 1 jednostka dystansu / 1 jednostka czasu
                potential_time = current_time + dist + service_time + dist_to_depot

                if current_capacity + demand <= max_capacity and potential_time <=
```

```

        if dist < min_dist:
            min_dist = dist
            nearest = c_id

    if nearest is None:
        break # Brak możliwości dodania kolejnego punktu do tej trasy

    # Aktualizacja stanu trasy
    route.append(nearest)
    current_capacity += customers[nearest][2]
    current_time += min_dist + customers[nearest][3]
    current_node = (customers[nearest][0], customers[nearest][1])
    unvisited.remove(nearest)

    routes.append(route)

return routes

# Przykład użycia:
depot_coords = (0, 0)
# format: {id: (x, y, demand, service_time)}
data = {1: (2, 3, 10, 0.5), 2: (5, 5, 15, 1.0), 3: (-1, 8, 5, 0.5)}
result = solve_cvrp_nn(depot_coords, data, 20, 12)
print(f"Trasy: {result}")

```

Kluczowe założenia

- **Ograniczenie czasowe:** Algorytm weryfikuje czas powrotu do bazy przed dodaniem klienta do trasy ($potential_time \leq 12$).
- **Pojemność:** Suma zapotrzebowania (*demand*) nie może przekroczyć *max_capacity*.
- **Złożoność:** Heurystyka ta jest szybka ($O(n^2)$), ale nie gwarantuje rozwiązania optymalnego.

8.3. Projekt bazy danych systemu kurierskiego w Django ORM

Poniższy projekt uwzględnia kluczowe relacje: przypisanie paczek do użytkowników, lokalizację magazynów, oraz logistyczne powiązanie tras z magazynami i przesyłkami.

Modele danych (models.py)

```

from django.db import models
from django.contrib.auth.models import AbstractUser

class User(AbstractUser):
    """Rozszerzony model użytkownika (klient, kurier, manager)"""
    ROLE_CHOICES = (
        ('customer', 'Klient'),

```



```

        ('courier', 'Kurier'),
        ('manager', 'Manager Magazynu'),
    )
    role = models.CharField(max_length=20, choices=ROLE_CHOICES)
    phone = models.CharField(max_length=15)

class Warehouse(models.Model):
    """Model magazynu / punktu przeładunkowego"""
    name = models.CharField(max_length=100)
    address = models.TextField()
    city = models.CharField(max_length=50)

    def __str__(self):
        return self.name

class Route(models.Model):
    """Model trasy przypisanej do kuriera i magazynu"""
    courier = models.ForeignKey(User, on_delete=models.SET_NULL,
                                null=True, related_name='routes')
    warehouse = models.ForeignKey(Warehouse, on_delete=models.CASCADE)
    start_time = models.DateTimeField()
    is_completed = models.BooleanField(default=False)

class Package(models.Model):
    """Model przesyłki z powiązaniami relacyjnymi"""
    tracking_number = models.CharField(max_length=50, unique=True)
    sender = models.ForeignKey(User, on_delete=models.CASCADE,
                                related_name='sent_packages')
    receiver_name = models.CharField(max_length=255)

    # Relacje kluczowe
    current_warehouse = models.ForeignKey(Warehouse, on_delete=models.SET_NULL,
                                           null=True, blank=True)
    current_route = models.ForeignKey(Route, on_delete=models.SET_NULL,
                                       null=True, blank=True, related_name='packages')

    STATUS_CHOICES = (
        ('in_transit', 'W transporcie'),
        ('in_warehouse', 'W magazynie'),
        ('delivered', 'Dostarczona'),
    )
    status = models.CharField(max_length=20, choices=STATUS_CHOICES)
    weight = models.DecimalField(max_digits=5, decimal_places=2)

```

Opis relacji i struktury

- **User** ↔ **Package**: Relacja One-to-Many (jeden użytkownik może wysłać wiele paczek).
- **Warehouse** ↔ **Route**: Relacja One-to-Many (trasa zawsze zaczyna się w konkretnym magazynie).
- **Route** ↔ **Package**: Relacja One-to-Many (trasa grupuje wiele paczek przypisanych do jednego kuriera w danym oknie czasowym).
- **User (Kurier)** ↔ **Route**: Relacja określająca, który pracownik jest odpowiedzialny za realizację konkretnego przejazdu.

8.4. Konfiguracja Docker Compose dla stosu Django, Next.js, PostgreSQL i Redis

Poniższy plik definiuje pełne środowisko uruchomieniowe, zapewniając izolację usług oraz komunikację wewnątrz sieci Dockerowej.

Plik docker-compose.yml

```
version: '3.8'

services:
  # Baza danych PostgreSQL
  db:
    image: postgres:15-alpine
    volumes:
      - postgres_data:/var/lib/postgresql/data
    env_file:
      - .env
    environment:
      - POSTGRES_DB=${DB_NAME}
      - POSTGRES_USER=${DB_USER}
      - POSTGRES_PASSWORD=${DB_PASSWORD}

  # Broker wiadomości Redis
  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"

  # Backend Django
  backend:
    build: ./backend
    command: python manage.py runserver 0.0.0.0:8000
    volumes:
      - ./backend:/app
    ports:
```

```

    - "8000:8000"
  environment:
    - DATABASE_URL=postgres://${DB_USER}:${DB_PASSWORD}@db:5432/${DB_NAME}
    - REDIS_URL=redis://redis:6379/0
  depends_on:
    - db
    - redis

# Frontend Next.js
frontend:
  build: ./frontend
  ports:
    - "3000:3000"
  volumes:
    - ./frontend:/app
    - /app/node_modules
  environment:
    - NEXT_PUBLIC_API_URL=http://backend:8000
  depends_on:
    - backend

volumes:
  postgres_data:

```

Kluczowe aspekty konfiguracji

- **Networking:** Docker Compose automatycznie tworzy sieć, w której usługi komunikują się po nazwach (np. backend łączy się z db: 5432).
- **Persystencja danych:** Użycie wolumenu `postgres_data` gwarantuje, że dane bazy nie zostaną utracone po zatrzymaniu kontenerów.
- **Zależności:** Dyrektywa `depends_on` zapewnia odpowiednią kolejność uruchamiania usług (najpierw baza i Redis, potem aplikacja).
- **Hot-reloading:** Dzięki podmontowaniu wolumenów lokalnych (`./backend:/app`), zmiany w kodzie są odzwierciedlane w czasie rzeczywistym bez przebudowy obrazu.

8.5. Grupowanie punktów GPS za pomocą algorytmu K-Means

Algorytm K-Means (K-średnich) jest jedną z najpopularniejszych metod uczenia nienadzorowanego, wykorzystywaną w logistyce do automatycznego podziału punktów (np. paczkomatów) na optymalne strefy dostaw.

Logika działania w logistyce

1. **Inicjalizacja:** Wybieramy liczbę k (liczba kurierów lub stref) oraz losujemy początkowe środki ciężkości (centroidy). 2. **Przypisanie:** Każdy punkt GPS zostaje przypisany do najbliższego centroidu na podstawie odległości euklidesowej. 3. **Aktualizacja:** Nowy środek strefy jest obliczany jako średnia współrzędnych

wszystkich przypisanych do niej punktów. 4. **Powtórzenie:** Proces trwa do momentu stabilizacji stref lub osiągnięcia limitu iteracji.

Implementacja w Python (scikit-learn)

```
import numpy as np
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Przykładowe współrzędne GPS paczkomatów (szerokość, długość)
points = np.array([
    [52.2297, 21.0122], [52.2350, 21.0150], [52.2400, 21.0300], # Warszawa Centrum
    [50.0647, 19.9450], [50.0610, 19.9370], [50.0700, 19.9550], # Kraków
    [51.1079, 17.0385], [51.1100, 17.0400], [51.1200, 17.0500] # Wrocław
])

# Definiujemy liczbę stref (np. 3 miasta/regiony)
num_zones = 3

# Model K-Means
kmeans = KMeans(n_clusters=num_zones, random_state=42, n_init=10)
kmeans.fit(points)

# Wyniki
labels = kmeans.labels_ # Indeks strefy dla każdego punktu
centroids = kmeans.cluster_centers_ # Centra logistyczne stref

print(f"Przypisanie punktów do stref: {labels}")
print(f"Współrzędne centrów stref: \n{centroids}")
```

Zastosowanie praktyczne

- **Optymalizacja lokalizacji magazynu:** Centroidy mogą wskazywać idealne miejsce na postawienie mikro-hubu przeładunkowego.
- **Zrównoważenie obciążenia:** Dobierając wagę punktów (np. liczba paczek), można modyfikować algorytm (Weighted K-Means), aby strefy były równe pod względem wolumenu, a nie tylko obszaru.
- **Ograniczenia:** Klasyczny K-Means używa odległości w linii prostej. W logistyce miejskiej warto rozważyć macierz czasu przejazdu (Routing Distance) zamiast euklidesowej.

8.6. Bezpieczne uwierzytelnianie w DRF z użyciem ciasteczek HttpOnly

Przechowywanie tokenów (np. JWT) w `localStorage` jest podatne na ataki XSS. Bezpieczniejszym podejściem jest przesyłanie tokena w ciasteczku z flagą `HttpOnly`, dzięki czemu skrypty JavaScript nie mają do niego dostępu.

1. Implementacja Custom Authentication (backend)

Musimy nadpisać domyślną klasę autoryzacji, aby DRF szukał tokena w ciasteczkach zamiast w nagłówku Authorization.

```
from rest_framework_simplejwt.authentication import JWTAuthentication
from rest_framework.response import Response

class CookieJWTAuthentication(JWTAuthentication):
    def authenticate(self, request):
        # Pobranie tokena z ciasteczka o nazwie 'access_token'
        raw_token = request.COOKIES.get('access_token')
        if raw_token is None:
            return None

        validated_token = self.get_validated_token(raw_token)
        return self.get_user(validated_token), validated_token
```

2. Logowanie i ustawianie ciasteczka

Widok logowania musi ręcznie ustawić ciasteczko w odpowiedzi HTTP.

```
from django.conf import settings
from rest_framework.views import APIView
from rest_framework_simplejwt.tokens import RefreshToken

class LoginView(APIView):
    def post(self, request):
        # (Tutaj walidacja użytkownika, np. authenticate())
        user = ...

        refresh = RefreshToken.for_user(user)
        response = Response({"message": "Zalogowano pomyślnie"})

        # Ustawienie ciasteczka HttpOnly
        response.set_cookie(
            key='access_token',
            value=str(refresh.access_token),
            httponly=True,
            secure=True,      # Wymaga HTTPS
            samesite='Lax'    # Ochrona przed CSRF
        )
        return response
```

Kluczowe aspekty bezpieczeństwa

- **HttpOnly:** Flaga uniemożliwia dostęp do ciasteczka poprzez `document.cookie`.
- **Secure:** Ciasteczko jest przesyłane wyłącznie przez szyfrowany protokół HTTPS.

- **SameSite (Lax/Strict):** Chroni przed atakami Cross-Site Request Forgery (CSRF), ograniczając wysyłanie ciasteczek w zapytaniach międzywitrynowych.
- **CSRF Token:** Przy użyciu ciasteczek do sesji, należy dodatkowo zaimplementować mechanizm `X-CSRFToken` dla metod modyfikujących dane (POST, PUT, DELETE).

Bibliografia

- [1] Vercel. *Next.js Documentation*, 2025. Dostęp: 2026-01-31. URL: <https://nextjs.org/docs>.
- [2] Encode OSS. *Django REST Framework - Toolkit for building Web APIs*, 2025. Dostęp: 2026-01-31. URL: <https://www.django-rest-framework.org/>.
- [3] Docker Inc. *Docker Documentation*, 2025. Dostęp: 2026-01-31. URL: <https://docs.docker.com/>.
- [4] The PostgreSQL Global Development Group. *PostgreSQL 16 Documentation*, 2025. Dostęp: 2026-01-31. URL: <https://www.postgresql.org/docs/>.
- [5] Celery Project. *Celery - Distributed Task Queue*, 2025. Dostęp: 2026-01-31. URL: <https://docs.celeryq.dev/>.
- [6] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Wprowadzenie do algorytmów*. Wydawnictwo Naukowe PWN, Warszawa, 2020. Rozdział dotyczący problemu komiwojażera (TSP).
- [7] J. MacQueen. Some methods for classification and analysis of multivariate observations. *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, 1:281–297, 1967.
- [8] GeoPy Contributors. *Geopy documentation - geocoding library for python*, 2025. Wykorzystano do obliczania dystansu Haversine'a. URL: <https://geopy.readthedocs.io/>.
- [9] DPD Polska. *Sieć punktów i automatów paczkowych dpd pickup*. <https://www.dpd.com/pl/pl/dpd-pickup/>, 2025. Dostęp: 2026-01-31.
- [10] Orlen S.A. *Orlen paczka - automaty i punkty odbioru*. <https://www.orlenpaczka.pl/>, 2025. Dostęp: 2026-01-31.

Spis rysunków

2.1	Diagram przypadków użycia systemu FasterPost	12
3.1	Diagram klas systemu FasterPost (widok logiczny)	14
3.2	Diagram związków encji (ERD) bazy danych systemu FasterPost	18
5.1	Harmonogram realizacji projektu FasterPost (Wykres Gantt)	23
7.1	Strona główna z nawigacją górną (Top Navigation).	29
7.2	Ekran logowania i rejestracji z polami tekstowymi.	30
7.3	Walidacja błędów w czasie rzeczywistym oraz ekran weryfikacji adresu e-mail.	30
7.4	Dashboard użytkownika z wykorzystaniem hierarchii wizualnej.	31
7.5	Proces nadawania przesyłki oraz widok śledzenia z wskaźnikiem postępu.	31
7.6	Ekran płatności oraz interfejs odbioru paczki.	31
7.7	Panel biznesowy z menu bocznym i statystykami.	32
7.8	Formularz masowego nadawania przesyłek dla sektora B2B.	32
7.9	Główny pulpit administratora systemu.	32
7.10	Moduł kontroli logistycznej z wizualizacją mapy.	33
7.11	Zarządzanie automatami paczkowymi oraz panel kuriera lokalnego.	33
7.12	Sekcja FAQ wykorzystująca wzorzec Accordion.	33

Spis tabel

1.1	Zestawienie wymagań funkcjonalnych	9
5.1	Macierz Śledzenia Wymagań (Requirements Traceability Matrix)	24
8.1	Zbiór najważniejszych promptów wykorzystanych w projekcie	36

Spis listingów

3.1	Struktura katalogów aplikacji frontendowej	19
4.1	Fragment algorytmu wyboru kolejnego celu w logistyce krajowej	20
4.2	Grupowanie przesyłek według stref	21
4.3	Implementacja wzoru Haversine'a	21
4.4	Logika dopasowania rozmiaru paczki do skrytki	22
5.1	Przykładowa konfiguracja pliku .env	25
5.2	Komenda uruchamiająca środowisko	25
5.3	Skrypty inicjalizujące dane testowe	26

Streszczenie

Celem niniejszej pracy inżynierskiej było zaprojektowanie i zaimplementowanie kompleksowego systemu logistycznego o nazwie **FasterPost**, wspierającego automatyzację procesów dostaw w modelu „Ostatniej Mili” (Last Mile) oraz transportu międzyhubowego (Line Haul). Projekt stanowi odpowiedź na rosnące zapotrzebowanie rynku e-commerce na wydajne systemy obsługi automatów paczkowych.

W ramach projektu wytworzono wielomodułową platformę internetową, umożliwiającą użytkownikom nadawanie i śledzenie przesyłek, dokonywanie płatności online oraz bezobsługowy odbiór paczek ze skrytek. Kluczowym elementem systemu jest moduł logistyczny przeznaczony dla kurierów, który automatycznie generuje optymalne trasy przejazdu, oraz panel administracyjny służący do zarządzania infrastrukturą i monitorowania stanu sieci. W warstwie inżynierskiej zaimplementowano własne rozwiązania algorytmiczne dla problemu komiwojażera (TSP) oraz problemu routingu pojazdów z ograniczeniami (CVRP), co pozwoliło na dynamiczną optymalizację łańcucha dostaw.

System został zrealizowany w architekturze klient-serwer. Warstwa serwerowa (Backend) powstała w języku **Python 3.12** z wykorzystaniem frameworka **Django** oraz **Django REST Framework**. Jako system zarządzania bazą danych wybrano **PostgreSQL**. Do obsługi zadań asynchronicznych i kolejkowania procesów logistycznych wykorzystano technologie **Celery** oraz **Redis**. Warstwa prezentacji (Frontend) została zaimplementowana w oparciu o framework **Next.js** (React) oraz bibliotekę **Tailwind CSS**, zapewniając responsywność i dostępność na urządzeniach mobilnych. Całość rozwiązania została konteneryzowana przy użyciu platformy Docker.

Załącznik nr 2 do Zarządzenia nr 228/2021 Rektora Uniwersytetu Rzeszowskiego z dnia 1 grudnia 2021 roku w sprawie ustalenia procedury antyplagiatowej w Uniwersytecie Rzeszowskim

OŚWIADCZENIE STUDENTA O SAMODZIELNOŚCI PRACY

.....Oleksii Nawrocki, Tomasz Nowak.....
Imię (imiona) i nazwisko studenta

Wydział Nauk Ścisłych i Technicznych

.....Informatyka.....
Nazwa kierunku

.....131400, 131478.....
Numer albumu

1. Oświadczam, że moja praca projektowa pt.: System paczkomatów i logistyki - FasterPost
 - 1) została przygotowana przeze mnie samodzielnie*,
 - 2) nie narusza praw autorskich w rozumieniu ustawy z dnia 4 lutego 1994 roku o prawie autorskim i prawach pokrewnych (t.j. Dz.U. z 2021 r., poz. 1062) oraz dóbr osobistych chronionych prawem cywilnym,
 - 3) nie zawiera danych i informacji, które uzyskałem/am w sposób niedozwolony,
 - 4) nie była podstawą otrzymania oceny z innego przedmiotu na uczelni wyższej ani mnie, ani innej osobie.
2. Jednocześnie wyrażam zgodę/nie wyrażam zgody** na udostępnienie mojej pracy projektowej do celów naukowo-badawczych z poszanowaniem przepisów ustawy o prawie autorskim i prawach pokrewnych.

(miejscowość, data)

(czytelny podpis studenta)

* Uwzględniając merytoryczny wkład prowadzącego przedmiot

** – niepotrzebne skreślić